

MongoDB Data Modeling — Complete Advanced Guide

Table of Contents

1. [What is Data Modeling in MongoDB?](#)
2. [Flexible Schema vs Fixed Schema](#)
3. [Core Principle — Store Together, Access Together](#)
4. [BSON Data Types](#)
5. [Document Relationships](#)
 - [One-to-One](#)
 - [One-to-Many](#)
 - [Many-to-Many](#)
6. [Embedding vs Referencing](#)
7. [Schema Design Patterns](#)
 - [Attribute Pattern](#)
 - [Bucket Pattern](#)
 - [Outlier Pattern](#)
 - [Computed Pattern](#)
 - [Subset Pattern](#)
 - [Extended Reference Pattern](#)
 - [Approximation Pattern](#)
 - [Tree Pattern](#)
 - [Polymorphic Pattern](#)
8. [Anti-Patterns to Avoid](#)
9. [Schema Validation](#)
10. [Polymorphic Data](#)
11. [Relational vs Document — Side-by-Side](#)
12. [Schema Design Process](#)
13. [Real-World Data Modeling Examples](#)
14. [Quick Reference Cheat Sheet](#)

1. What is Data Modeling?

Data modeling is the process of **defining how data is organized, stored, and related** within a database to best serve your application's access patterns.

In MongoDB, data modeling revolves around **one central question**:

How does my application ACCESS this data?

Unlike relational databases that model data around **storage normalization**, MongoDB models data around **query performance**.

Relational Mindset:

"How do I reduce
data duplication?"

- Normalize into tables
- Join at query time

MongoDB Mindset:

"How does my application
read and write this data?"

- Store together what's
accessed together

2. Flexible Schema vs Fixed Schema

MongoDB uses a **flexible schema** (also called schema-less or dynamic schema).

What Flexibility Means

```
// All 3 documents can coexist in the SAME collection ✓  
db.products.insertMany([  
  // Product 1: Simple item  
  { _id: 1, name: "Pen", price: 2.50, color: "blue" },  
  
  // Product 2: Electronics with extra fields  
  { _id: 2, name: "Laptop", price: 999, brand: "Dell",  
    specs: { ram: "16GB", storage: "512GB SSD", cpu: "i7" } },  
  
  // Product 3: Clothing with completely different fields  
  { _id: 3, name: "T-Shirt", price: 25,  
    sizes: ["S", "M", "L", "XL"], material: "Cotton", gender: "Unisex" }  
])
```

Iterative Schema Evolution

```
// Version 1: Basic user
{ _id: 1, name: "Alice", email: "alice@example.com" }

// Version 2: Added phone (no migration needed for old docs!)
{ _id: 2, name: "Bob", email: "bob@example.com", phone: "555-1234" }

// Version 3: Restructured address (old and new formats coexist)
{ _id: 3, name: "Carol", email: "carol@example.com",
  address: { city: "Hyderabad", state: "Telangana", zip: "500001" } }

// ✅ All 3 live in the same "users" collection without migration
```

Flexibility vs Governance



What is a Workload?

A workload is the **complete set of operations** your application runs against the database — every read, write, update, and delete — along with how often each runs.

$$\text{Workload} = \text{Operations} \times \text{Frequency} \times \text{Priority}$$

Building Your Workload Table

Fill out this table for **every significant operation** in your application.

Action	Query Type	Information	Frequency	Priority
User describes what triggers the action	read or write	Fields written or returned	Estimated operations/sec or /day	high , medium , or low

Workload Table — Full Example (E-Commerce App)

Action	Query Type	Fields Involved	Frequency	Priority
User logs in	read	email , passwordHash , status	5,000 / hour	high
View product page	read	name , price , images , specs , recentReviews , stats	50,000 / hour	high
Search products by name/category	read	name , category , price , status	30,000 / hour	high
Add item to cart	write	userId , items[] , updatedAt	8,000 / hour	high
Place order	write	userId , items[] , total , status , address	2,000 / hour	high
Admin: update product price	write	_id , price , updatedAt	100 / hour	medium

Action	Query Type	Fields Involved	Frequency	Priority
User views order history	read	userId , status , createdAt , items[] , total	3,000 / hour	medium
User submits a review	write	productId , userId , rating , body , createdAt	500 / hour	medium
Admin: generate sales report	read (agg)	createdAt , total , category	10 / day	low
System: expire old sessions	delete	expiresAt	continuous	medium

Analyzing Your Workload Table

Once filled, extract these insights:

HIGH frequency + HIGH priority → These MUST have indexes and optimized schemas

HIGH frequency + LOW priority → Index but less urgency

LOW frequency + HIGH priority → Optimize for correctness, not just speed

LOW frequency + LOW priority → OK to use collection scans or \$lookup

// Example insight from table above:

// "View product page" = 50,000 reads/hour → HIGH priority

// → All fields it returns should be in ONE document (embed reviews, specs, images)

// → Should never require a \$lookup

// "Admin: generate sales report" = 10 reads/day → LOW priority

// → Can use aggregation pipeline with \$lookup – latency is acceptable

Using the Database Profiler

For **existing applications**, use MongoDB's built-in profiler to discover actual query patterns.

```

// Enable profiling – log all operations slower than 100ms
db.setProfilingLevel(1, { slowms: 100 })

// Enable profiling – log ALL operations (development only!)
db.setProfilingLevel(2)

// Disable profiling
db.setProfilingLevel(0)

// Read the profiler output
db.system.profile.find().sort({ ts: -1 }).limit(20).pretty()

// Find the most time-consuming queries
db.system.profile.find().sort({ millis: -1 }).limit(10)

// Find the most frequent slow queries
db.system.profile.aggregate([
  { $group: { _id: "$op", count: { $sum: 1 }, avgMs: { $avg: "$millis" } } },
  { $sort: { count: -1 } }
])

```

Key fields in profiler output:

```

{
  op: "query",           // operation type: query, insert, update, remove
  ns: "mydb.products",   // namespace
  command: { find: "products", ... }, // the actual query
  keysExamined: 5,        // index keys scanned
  docsExamined: 5,        // documents scanned
  nreturned: 3,           // documents returned
  millis: 12,             // execution time in ms
  planSummary: "IXSCAN { name: 1 }", // plan used
  ts: ISODate("2024-01-15T10:30:00Z")
}
// ⚠️ If docsExamined >> nreturned → missing or inefficient index

```

3. Step 2 — Map Schema Relationships

Determine **how data entities relate** to each other and whether to **embed** or **reference** related data.

3. Core Principle

Data that is accessed together should be stored together.

This single principle drives every MongoDB schema decision.

Ask these 3 questions before modeling any entity:

1. What queries does the app run on this data?
2. What data is always fetched together?
3. What is the read-to-write ratio of this data?

Example: E-Commerce Product Page

// ❌ Relational thinking – 5 queries to load one page:

```
SELECT * FROM products WHERE id = 1;
SELECT * FROM product_images WHERE product_id = 1;
SELECT * FROM product_specs WHERE product_id = 1;
SELECT * FROM reviews WHERE product_id = 1 LIMIT 5;
SELECT * FROM categories WHERE id = product.category_id;
```

// ✅ MongoDB thinking – 1 query loads everything:

```
db.products.findOne({ _id: 1 })
```

// Result:

```
{
  _id: 1,
  name: "Wireless Headphones",
  price: 149.99,
  category: { id: "c1", name: "Electronics" },
  images: ["img1.jpg", "img2.jpg", "img3.jpg"],
  specs: { battery: "20hrs", connectivity: "Bluetooth 5.0" },
  recentReviews: [
    { user: "Alice", rating: 5, comment: "Amazing!" },
    { user: "Bob", rating: 4, comment: "Great value" }
  ],
  reviewCount: 238,
  avgRating: 4.6
}
```

4. BSON Data Types

MongoDB stores data as **BSON (Binary JSON)** — a superset of JSON with additional types.

BSON Type	Example	Notes
String	"hello"	UTF-8 encoded
Integer (32)	42	NumberInt(42)
Integer (64)	NumberLong(9999999999)	For large numbers

BSON Type	Example	Notes
Double	3.14	64-bit float
Decimal128	NumberDecimal("19.99")	✅ Use for money/finance
Boolean	true / false	
Date	new Date()	Stored as UTC milliseconds
ObjectId	ObjectId("507f1f77bcf86cd799439011")	12-byte auto-generated _id
Array	["a", "b", "c"]	Ordered list of values
Object	{ city: "Hyderabad" }	Embedded document
Null	null	Explicit null value
Binary	BinData(0, "...")	For files, images
Regex	/pattern/flags	Regular expressions
Timestamp	Timestamp(1, 1)	Internal MongoDB use
MinKey/MaxKey	Special BSON values	For index boundary comparisons

```
// Example document using diverse BSON types
db.orders.insertOne({
  _id: ObjectId(),           // ObjectId
  orderId: "ORD-2024-001",   // String
  userId: NumberLong(123456789), // Int64
  total: NumberDecimal("199.99"), // Decimal128 – exact money
  isPriority: true,          // Boolean
  createdAt: new Date(),     // Date
  tags: ["express", "fragile"], // Array
  shipping: {                // Embedded Object
    address: "123 Main St",
    city: "Hyderabad",
    pincode: "500001"
  },
  notes: null                // Null
})
```

⚠️ Always use `NumberDecimal` for financial values — regular JS floats cause rounding errors.

5. Document Relationships

One-to-One

Each document maps to exactly one other document.

```
// Example: Patient ↔ Medical Record

// Option A: Embed (recommended when always accessed together)
db.patients.insertOne({
  _id: 1,
  name: "Alice Kumar",
  dob: new Date("1990-05-15"),
  medicalRecord: {                                // ← embedded one-to-one
    bloodType: "O+",
    allergies: ["penicillin"],
    lastVisit: new Date("2024-01-10"),
    conditions: ["hypertension"]
  }
})

// Option B: Reference (recommended when accessed separately or very large)
db.patients.insertOne({ _id: 1, name: "Alice Kumar" })
db.medicalRecords.insertOne({ _id: 1, patientId: 1, bloodType: "O+", ... })
```

One-to-Many

One document is associated with multiple related documents.

```
// Example: User → Posts

// Option A: Embed array (good for bounded, small lists)
db.users.insertOne({
  _id: 1,
  name: "Alice",
  posts: [                                     // ← embedded array (OK if < 20 items)
    { title: "MongoDB Tips", date: new Date(), likes: 42 },
    { title: "Node.js Guide", date: new Date(), likes: 18 }
  ]
})

// Option B: Reference in child (better for unbounded lists)
db.users.insertOne({ _id: 1, name: "Alice" })
db.posts.insertMany([
  { _id: 101, userId: 1, title: "MongoDB Tips", likes: 42 },
  { _id: 102, userId: 1, title: "Node.js Guide", likes: 18 },
  // ... could be thousands of posts
])
// Query: db.posts.find({ userId: 1 })

// Option C: Reference in parent (for small, fixed lists)
db.users.insertOne({
  _id: 1,
  name: "Alice",
  recentPostIds: [101, 102, 103]           // ← array of references
})
```

One-to-Many Decision Guide

How many "many" items?	Grow unbounded?	Model
Few (< 20)	No	→ Embed array
Moderate (< 1000)	No	→ Reference in child
Unbounded / Large	Yes	→ Reference in child
Need top-N frequently	Yes	→ Subset pattern (embed top-N only)

Many-to-Many

Each document relates to multiple on both sides.

```
// Example: Students ↔ Courses
```

```
// Option A: Array of references in both documents
```

```
db.students.insertOne({
  _id: "s1",
  name: "Alice",
  enrolledCourseIds: ["c1", "c2", "c3"]
})
```

```
db.courses.insertOne({
  _id: "c1",
  title: "MongoDB Advanced",
  enrolledStudentIds: ["s1", "s2", "s3"]
})
```

```
// Option B: Junction collection (for rich relationship data)
```

```
db.enrollments.insertMany([
  {
    studentId: "s1",
    courseId: "c1",
    enrolledAt: new Date("2024-01-01"),
    grade: "A",
    progress: 87
  },
  {
    studentId: "s1",
    courseId: "c2",
    enrolledAt: new Date("2024-02-01"),
    grade: null,
    progress: 45
  }
])
// Query: db.enrollments.find({ studentId: "s1" })
```

6. Embedding vs Referencing

This is the most important decision in MongoDB schema design.

Embedding (Denormalization)

Store related data **inside** the parent document.

```
// Embedded order with items
db.orders.insertOne({
  _id: "o1",
  userId: "u1",
  status: "shipped",
  createdAt: new Date(),
  shippingAddress: {                // ← embedded object
    street: "123 Main St",
    city: "Hyderabad",
    state: "Telangana"
  },
  items: [                          // ← embedded array
    { sku: "LAPTOP-01", name: "Laptop Pro", qty: 1, price: 999 },
    { sku: "MOUSE-05", name: "Wireless Mouse", qty: 2, price: 29 }
  ],
  total: 1057
})
```

✅ Use Embedding When:

- Data is always accessed together
- Data belongs to one parent (ownership relationship)
- One-to-one or one-to-few relationships
- Subdocument rarely changes independently
- Data is small and doesn't grow unboundedly
- Read performance is critical (avoid joins)

Referencing (Normalization)

Store a pointer (`_id`) to a document in another collection.

```
// Referenced: order items in separate collection
db.orders.insertOne({
  _id: "o1",
  userId: "u1",
  itemIds: ["item_101", "item_102"] // ← references
})

db.orderItems.insertMany([
  { _id: "item_101", orderId: "o1", sku: "LAPTOP-01", qty: 1, price: 999 },
  { _id: "item_102", orderId: "o1", sku: "MOUSE-05", qty: 2, price: 29 }
])
```

✅ Use Referencing When:

- Data is accessed independently of parent
- One-to-many with large/unbounded "many" side
- Data is shared across multiple parent documents
- Data changes frequently and independently
- Document would exceed 16MB BSON limit
- Many-to-many relationships

Embedding vs Referencing Decision Matrix

Factor	Embed ✅	Reference ✅
Access pattern	Always together	Often separate
Relationship cardinality	One-to-Few	One-to-Many / M2M
Data ownership	Owned by parent	Shared across docs
Update frequency	Rare	Frequent independent
Document size concern	Small subdoc	Could be large
Duplication tolerance	OK to duplicate	Needs single source
Query complexity	Simple reads	Uses \$lookup

7. Schema Design Patterns

Attribute Pattern

When documents have **many similar fields** or a large number of fields that only some documents share. Converts key-value pairs into a searchable array.

Problem:

```
// Each product has different specs – hard to index all of them
{ _id: 1, voltage: "220V", wattage: "60W", color: "white" }
{ _id: 2, size: "XL", material: "Cotton", fit: "Slim" }
{ _id: 3, processor: "i7", ram: "16GB", storage: "512GB" }
// ❌ Requires separate index for every spec field
```

Solution:

```
// Convert to attribute array – one index covers all specs
db.products.insertOne({
  _id: 1,
  name: "Smart Bulb",
  attributes: [
    { key: "voltage", value: "220V", unit: "V" },
    { key: "wattage", value: "60", unit: "W" },
    { key: "color", value: "white" }
  ]
})

// ONE index covers all attributes
db.products.createIndex({ "attributes.key": 1, "attributes.value": 1 })

// Search any attribute efficiently
db.products.find({ attributes: { $elemMatch: { key: "voltage", value: "220V" } } })
```

Bucket Pattern

Groups time-series or sequential data into **buckets** (chunks) instead of one document per event.
Dramatically reduces document count.

Problem (one doc per event):

```
// IoT sensor creates millions of documents
{ sensorId: "s1", timestamp: ISODate("2024-01-01T00:00:01"), temp: 22.1 }
{ sensorId: "s1", timestamp: ISODate("2024-01-01T00:00:02"), temp: 22.3 }
{ sensorId: "s1", timestamp: ISODate("2024-01-01T00:00:03"), temp: 22.2 }
// ❌ 86,400 documents per sensor per day
```

Solution (bucket by hour):

```
db.sensorData.insertOne({
  sensorId: "s1",
  date: ISODate("2024-01-01T00:00:00"),
  type: "temperature",
  count: 60,
  minTemp: 21.8,
  maxTemp: 22.9,
  avgTemp: 22.3,
  readings: [
    { t: 0, v: 22.1 },
    { t: 1, v: 22.3 },
    { t: 2, v: 22.2 },
    // ... up to 60 readings per bucket
  ]
})
// ✅ 24 documents per sensor per day instead of 86,400
```

Outlier Pattern

Handles rare documents that **exceed normal size bounds** (e.g., a viral post with millions of followers).


```
// Normal user
db.users.insertOne({
  _id: "u1",
  name: "Regular User",
  followers: ["u2", "u3", "u4"],    // small list
  hasOverflow: false
})

// Celebrity/viral user – followers would exceed 16MB
db.users.insertOne({
  _id: "celebrity1",
  name: "Movie Star",
  followers: ["u1", "u2", /* first 1000 */],
  hasOverflow: true                // ← flag for outlier
})

// Overflow stored in separate collection
db.followerOverflow.insertMany([
  { userId: "celebrity1", followers: ["u1001", "u1002", /* next batch */] },
  { userId: "celebrity1", followers: ["u2001", "u2002", /* next batch */] }
])

// Application logic:
// if (user.hasOverflow) → also query followerOverflow collection
```

Computed Pattern

Pre-computes and stores **expensive aggregation results** in the document, updated on write.

Problem:

```
// Calculating revenue stats on every request = expensive
db.orders.aggregate([
  { $match: { productId: "p1" } },
  { $group: { _id: null, total: { $sum: "$amount" }, count: { $sum: 1 }, avg: { $avg: "$amount" } }
])
// ❌ Scans all orders every page load
```

Solution:

```

// Pre-computed stats stored directly on the product
db.products.insertOne({
  _id: "p1",
  name: "Laptop Pro",
  price: 999,
  stats: {                                     // ← pre-computed, updated on each sale
    totalRevenue: 49950,
    totalSales: 50,
    avgOrderValue: 999,
    lastUpdated: new Date()
  }
})

// On each new order, update stats atomically
db.products.updateOne(
  { _id: "p1" },
  {
    $inc: {
      "stats.totalRevenue": 999,
      "stats.totalSales": 1
    },
    $set: { "stats.lastUpdated": new Date() }
  }
)
// ✅ Stats are instant reads – no aggregation needed

```

Subset Pattern

For one-to-many relationships, embed only the **most-needed subset** (e.g., latest 5 reviews) and store the full list separately.

```
// Product document – embed ONLY top 5 reviews for quick display
db.products.insertOne({
  _id: "p1",
  name: "Wireless Headphones",
  reviewCount: 238,
  avgRating: 4.6,
  recentReviews: [ // ← subset: only latest 5
    { user: "Alice", rating: 5, comment: "Amazing!", date: new Date() },
    { user: "Bob", rating: 4, comment: "Great value", date: new Date() },
    // ... 3 more
  ]
})

// All reviews live in their own collection for "View All Reviews" page
db.reviews.insertMany([
  { productId: "p1", user: "Alice", rating: 5, comment: "Amazing!", date: new Date() },
  { productId: "p1", user: "Bob", rating: 4, comment: "Great value", date: new Date() },
  // ... 238 total
])

// Update: on new review, push to subset if top-5 AND insert into reviews collection
```

Extended Reference Pattern

When referencing another document, **duplicate a few frequently-needed fields** to avoid a join for common queries.

```
// ❌ Pure reference – requires $lookup for every order display
db.orders.insertOne({ _id: "o1", userId: "u1", total: 299 })
// Every query needs: db.users.findOne({ _id: "u1" }) to get user name

// ✅ Extended reference – embed only the fields you need most
db.orders.insertOne({
  _id: "o1",
  userId: "u1",
  // Duplicated from user – OK because rarely changes
  userName: "Alice Kumar",           // ← extended reference fields
  userEmail: "alice@example.com",
  shippingAddress: { city: "Hyderabad", state: "Telangana" },
  total: 299
})
// ✅ Order list page works without any join
```

⚠️ Only duplicate **stable, rarely-changing** fields. If the user changes their name, update strategy needed.

Approximation Pattern

For high-write counters (views, clicks, impressions) where **exact accuracy isn't critical**, reduce writes by approximating.

```
// ❌ Update view count on every single page view – millions of writes
db.articles.updateOne({ _id: "a1" }, { $inc: { views: 1 } }) // on every request

// ✅ Only write to DB every ~100 views (approximate)
// Application logic:
const threshold = 100
if (Math.random() < 1 / threshold) {
  db.articles.updateOne(
    { _id: "a1" },
    { $inc: { views: threshold } } // add 100 at a time
  )
}
// ✅ 100x fewer write operations – acceptable for "view count" display
```

Tree Pattern

Models **hierarchical data** (org charts, categories, comments) in documents.

Array of Ancestors (most flexible)

```
db.categories.insertMany([
  { _id: "electronics",    name: "Electronics",    ancestors: [],
    { _id: "computers",    name: "Computers",      ancestors: ["electronics"],
    { _id: "laptops",      name: "Laptops",        ancestors: ["electronics", "computers"],
    { _id: "gaming-laptops", name: "Gaming Laptops", ancestors: ["electronics", "computers", "laptops"]
  ])

// ✅ Find ALL descendants of "computers"
db.categories.find({ ancestors: "computers" })

// ✅ Find ALL ancestors of "gaming-laptops"
db.categories.findOne({ _id: "gaming-laptops" }).ancestors
```

Materialized Path

```
db.categories.insertMany([
  { _id: "electronics",    path: "Electronics" },
  { _id: "computers",      path: "Electronics,Computers" },
  { _id: "laptops",        path: "Electronics,Computers,Laptops" }
])

// Find all under Electronics using regex
db.categories.find({ path: /^Electronics/ })
db.categories.createIndex({ path: 1 })
```

Polymorphic Pattern

Store **different entity types** with varying schemas in the same collection, unified by a type discriminator field.

```
// All vehicles in one collection – different fields per type
db.vehicles.insertMany([
  {
    _id: 1,
    type: "car",
    make: "Toyota", model: "Camry", year: 2022,
    doors: 4, fuelType: "Hybrid", trunkVolume: "428L"
  },
  {
    _id: 2,
    type: "motorcycle",
    make: "Honda", model: "CBR600", year: 2023,
    engineCC: 600, style: "Sport", hasSidecar: false
  },
  {
    _id: 3,
    type: "truck",
    make: "Ford", model: "F-150", year: 2023,
    payload: "1000kg", bedLength: "5.5ft", towCapacity: "5000kg"
  }
])

// Query specific type
db.vehicles.find({ type: "car", year: { $gte: 2020 } })

// Query across all types (shared fields)
db.vehicles.find({ make: "Honda" })

// Create index for discriminator
db.vehicles.createIndex({ type: 1, make: 1 })
```

8. Anti-Patterns

✗ Massive Arrays (Unbounded Growth)

```
// ✗ BAD – array grows forever, hits 16MB limit
db.users.updateOne(
  { _id: "u1" },
  { $push: { allActivityLogs: { action: "login", date: new Date() } } }
)
// After years: document has millions of items, 16MB BSON limit reached

// ✓ GOOD – separate collection with reference
db.activityLogs.insertOne({ userId: "u1", action: "login", date: new Date() })
```

✗ Unnecessary Indexes

```
// ✗ BAD – indexing fields that are never queried
db.users.createIndex({ bio: 1 })           // bio is never filtered
db.users.createIndex({ profilePicUrl: 1 }) // never queried
db.users.createIndex({ createdAt: 1 })     // only queried with userId

// ✓ GOOD – index only queried fields, use compound index
db.users.createIndex({ userId: 1, createdAt: -1 })
```

❌ Bloated Documents

```
// ❌ BAD – storing everything in one document
db.products.insertOne({
  _id: "p1",
  name: "Laptop",
  allReviews: [ /* 10,000 reviews */ ],      // huge embedded array
  fullDescription: "... 50,000 characters ...", // massive text
  auditHistory: [ /* years of changes */ ]    // unbounded history
})
```

```
// ✅ GOOD – subset pattern + separate collections
db.products.insertOne({
  _id: "p1",
  name: "Laptop",
  recentReviews: [ /* top 5 only */ ],      // subset
  descriptionSummary: "... 200 chars ...",  // summary only
  reviewCount: 10000
})
```

❌ Separating Data That's Always Accessed Together

```
// ❌ BAD – normalized like SQL, causes unnecessary $lookup
db.users.insertOne({ _id: 1, name: "Alice" })
db.userProfiles.insertOne({ userId: 1, bio: "Developer", avatar: "..." })
db.userSettings.insertOne({ userId: 1, theme: "dark", notifications: true })
```

// Every profile page = 3 queries + \$lookup

```
// ✅ GOOD – embed data accessed together
db.users.insertOne({
  _id: 1,
  name: "Alice",
  profile: { bio: "Developer", avatar: "..." },
  settings: { theme: "dark", notifications: true }
})
```


✗ Using MongoDB Like a Relational DB

```
// ✗ BAD – over-normalized, requires many joins  
db.orders.find() → join → db.orderItems → join → db.products → join → db.users  
  
// ✓ GOOD – embed what belongs together, reference what doesn't  
db.orders.findOne({ _id: "o1" })  
// Returns everything needed for an order page in one read
```

9. Schema Validation

Enforce rules on documents using **JSON Schema validation**.

```
// Add validation when creating a collection
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "createdAt"],
      properties: {
        name: {
          bsonType: "string",
          minLength: 2,
          maxLength: 100,
          description: "Must be a string between 2-100 characters"
        },
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
          description: "Must be a valid email address"
        },
        age: {
          bsonType: "int",
          minimum: 0,
          maximum: 120
        },
        role: {
          enum: ["admin", "user", "moderator"],
          description: "Must be one of the allowed roles"
        },
        address: {
          bsonType: "object",
          required: ["city", "country"],
          properties: {
            city: { bsonType: "string" },
            country: { bsonType: "string" },
            zip: { bsonType: "string" }
          }
        },
        tags: {
          bsonType: "array",
          items: { bsonType: "string" },
          uniqueItems: true
        },
        createdAt: {
          bsonType: "date"
        }
      }
    }
  }
})
```

```

    }
  }
},
validationLevel: "moderate",    // "strict" = validate all, "moderate" = skip existing invalid
validationAction: "error"      // "error" = reject invalid, "warn" = log but allow
}))

```

Add Validation to Existing Collection

```

db.runCommand({
  collMod: "users",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email"],
      properties: {
        email: { bsonType: "string" }
      }
    }
  },
  validationLevel: "moderate",
  validationAction: "error"
}))

```

Bypass Validation (Admin Operations)

```

// Only available with bypassDocumentValidation
db.runCommand({
  insert: "users",
  documents: [{ name: "Migration Doc" }],
  bypassDocumentValidation: true
}))

```

10. Polymorphic Data

Store **different shapes of data** in one collection using a type discriminator — great for product catalogs, content management, and event sourcing.

```
// Content Management System – all content types in one collection
```

```
db.content.insertMany([
  {
    _id: "c1",
    contentType: "article",          // ← discriminator field
    title: "MongoDB Guide",
    body: "Full article text...",
    author: "Alice",
    tags: ["mongodb", "database"],
    readTime: "8 min",
    publishedAt: new Date()
  },
  {
    _id: "c2",
    contentType: "video",
    title: "MongoDB Tutorial",
    videoUrl: "https://youtube.com/...",
    duration: 1240,                  // seconds
    thumbnailUrl: "thumb.jpg",
    transcriptAvailable: true,
    publishedAt: new Date()
  },
  {
    _id: "c3",
    contentType: "podcast",
    title: "Database Design Episode",
    audioUrl: "https://podcast.com/...",
    duration: 2700,
    episodeNumber: 42,
    showNotes: "In this episode...",
    publishedAt: new Date()
  }
])
```

```
// Query all content by date (polymorphic)
```

```
db.content.find().sort({ publishedAt: -1 }).limit(20)
```

```
// Query only videos
```

```
db.content.find({ contentType: "video" })
```

```
// Index for type-based queries
```

```
db.content.createIndex({ contentType: 1, publishedAt: -1 })
```

11. Relational vs Document — Side-by-Side

FEATURE	RELATIONAL (SQL)	DOCUMENT (MongoDB)
Schema	Fixed – define before insert	Flexible – evolves freely
Joins	JOIN across tables	\$lookup or embedding
Normalization	Encouraged (3NF)	Denormalization preferred
Relationships	Foreign keys	Embedding or _id references
Transactions	Native, row-level	Supported (4.0+ multi-doc)
Scaling	Vertical (scale up)	Horizontal (sharding)
Data shape	Tables – rows & columns	Documents – JSON-like BSON
Query language	SQL	MQL (MongoDB Query Language)
Schema evolution	ALTER TABLE (disruptive)	Add field (non-disruptive)

Data Model Comparison

```
// SQL – 4 normalized tables
CREATE TABLE users (id INT, name VARCHAR(100));
CREATE TABLE orders (id INT, user_id INT, total DECIMAL);
CREATE TABLE order_items (id INT, order_id INT, sku VARCHAR);
CREATE TABLE products (sku VARCHAR, name VARCHAR, price DECIMAL);
-- Query requires 3 JOINS

// MongoDB – 1 document per order
{
  _id: "o1",
  user: { id: "u1", name: "Alice" },           // extended reference
  items: [
    { sku: "LAP-01", name: "Laptop", qty: 1, price: 999 },
    { sku: "MOU-05", name: "Mouse", qty: 2, price: 29 }
  ],
  total: 1057,
  createdAt: new Date()
}
```

// Query: one document fetch – zero joins

12. Schema Design Process

Follow this structured process before writing any production code.

STEP 1: Identify Entities

List all the "things" in your system:

→ User, Product, Order, Review, Category, Inventory...

STEP 2: Map Access Patterns (most important step!)

For each entity, answer:

- What queries run on this data? (find by X, sort by Y)
- What data is always fetched together?
- Read frequency vs write frequency?
- How large can this data grow?

STEP 3: Define Relationships

- One-to-One? → Likely embed
- One-to-Few? → Likely embed array
- One-to-Many? → Reference in child
- Many-to-Many? → Junction collection or arrays

STEP 4: Choose Embed vs Reference

Use the Decision Matrix from Section 6

STEP 5: Apply Relevant Patterns

- Large arrays? → Bucket or Subset pattern
- Many sparse fields? → Attribute pattern
- Cached aggregates? → Computed pattern
- Deep hierarchy? → Tree pattern
- Multiple shapes? → Polymorphic pattern

STEP 6: Add Schema Validation

- Define required fields
- Validate field types
- Enforce enum values
- Set string length limits

STEP 7: Define Indexes

- Index fields used in \$match
- Apply ESR rule to compound indexes

- Add TTL index for expiring data
- Add unique index for natural keys (email, sku)

STEP 8: Test & Iterate

- Run `explain("executionStats")` on all critical queries
- Monitor for COLLSCAN in production
- Evolve schema as access patterns change

13. Real-World Examples

E-Commerce Platform

```
// Product catalog – polymorphic with attribute pattern
db.products.insertOne({
  _id: "prod-001",
  type: "electronics",
  name: "Sony WH-1000XM5",
  sku: "SONY-WH-XM5-BLK",
  price: NumberDecimal("349.99"),
  brand: "Sony",
  status: "active",

  // Subset pattern – only 5 most recent reviews
  recentReviews: [
    { user: "Alice", rating: 5, body: "Best headphones!", date: new Date() }
  ],
  // Computed pattern – pre-aggregated stats
  reviewStats: { count: 1247, avgRating: 4.7 },

  // Attribute pattern – flexible specs per product type
  attributes: [
    { key: "batteryLife",    value: "30",      unit: "hours" },
    { key: "connectivity",   value: "Bluetooth 5.2" },
    { key: "noiseCancelling", value: "true" },
    { key: "weight",         value: "250",     unit: "g" }
  ],

  category: { id: "cat-audio", name: "Audio", path: "Electronics,Audio,Headphones" },
  images: ["main.jpg", "side.jpg", "in-use.jpg"],
  inventory: { quantity: 48, reserved: 3, warehouse: "WH-HYD" },
  createdAt: new Date(),
  updatedAt: new Date()
})
```

Social Media User Profile

```
db.users.insertOne({
  _id: "user-alice",
  username: "alice_dev",
  email: "alice@example.com",
  passwordHash: "$2b$10$...",
  profile: {
    displayName: "Alice Kumar",
    bio: "MongoDB enthusiast | Node.js developer",
    avatar: "https://cdn.example.com/alice.jpg",
    location: "Hyderabad, India",
    website: "https://alice.dev"
  },
  stats: {                                // computed pattern
    postsCount: 142,
    followersCount: 3821,
    followingCount: 287
  },
  // Subset: only last 10 notifications in document
  recentNotifications: [
    { type: "like", from: "bob_dev", postId: "p1", at: new Date() }
  ],
  settings: {
    theme: "dark",
    emailNotifications: true,
    privacy: "public"
  },
  joinedAt: new Date(),
  lastActiveAt: new Date()
})
```



IoT Sensor Data (Bucket Pattern)

```
// Bucket: one document per sensor per hour
db.sensorReadings.insertOne({
  _id: { sensorId: "sensor-01", hour: ISODate("2024-01-15T14:00:00Z") },
  sensorId: "sensor-01",
  location: { building: "A", floor: 3, room: "3B" },
  type: "temperature",
  hour: ISODate("2024-01-15T14:00:00Z"),
  count: 60,
  stats: {
    min: 21.2, max: 23.8, avg: 22.5, sum: 1350
  },
  readings: [
    { minute: 0, value: 22.1 },
    { minute: 1, value: 22.4 },
    // ... 60 readings per hour
  ]
})

// Query: average temperature for sensor-01 on Jan 15
db.sensorReadings.find({
  sensorId: "sensor-01",
  hour: {
    $gte: ISODate("2024-01-15T00:00:00Z"),
    $lt: ISODate("2024-01-16T00:00:00Z")
  }
})
```



Learning Management System (Complex Relationships)

```
// Course document – extended reference + subset patterns
db.courses.insertOne({
  _id: "course-mongodb-101",
  title: "MongoDB for Developers",
  slug: "mongodb-for-developers",
  instructor: {                                // extended reference – stable fields only
    id: "inst-001",
    name: "Dr. Ravi Sharma",
    avatarUrl: "ravi.jpg",
    title: "Senior Database Engineer"
  },
  stats: {                                     // computed pattern
    enrolledCount: 4521,
    completionRate: 73,
    avgRating: 4.8,
    reviewCount: 892
  },
  curriculum: [                               // embedded – always displayed with course
    {
      sectionId: "s1",
      title: "Introduction",
      lessons: [
        { lessonId: "l1", title: "What is MongoDB?", duration: 420, type: "video" },
        { lessonId: "l2", title: "Installing MongoDB", duration: 360, type: "video" },
        { lessonId: "l3", title: "Quiz 1", type: "quiz", questions: 10 }
      ]
    }
  ],
  topReviews: [ /* subset: top 3 */ ],
  tags: ["mongodb", "nosql", "database", "backend"],
  price: NumberDecimal("49.99"),
  publishedAt: new Date()
})

// Enrollment – junction collection (rich M2M relationship)
db.enrollments.insertOne({
  _id: ObjectId(),
  userId: "user-alice",
  courseId: "course-mongodb-101",
  enrolledAt: new Date(),
  progress: {
```

```
    completedLessons: ["11", "12"],  
    currentLesson: "13",  
    percentComplete: 35,  
    lastAccessedAt: new Date()  
  },  
  certificate: null,  
  paymentId: "pay-xyz"  
})
```

14. Cheat Sheet

RELATIONSHIP	CARDINALITY	RECOMMENDED APPROACH
User → Address	One-to-One	Embed address in user
Patient → Record	One-to-One	Embed (if always accessed together)
Blog → Comments	One-to-Many	Reference (comments collection)
Order → Items	One-to-Few	Embed items array in order
User → Orders	One-to-Many	Reference (orders collection)
Student → Courses	Many-to-Many	Junction collection (enrollments)
Product → Reviews	One-to-Many	Subset (embed top 5) + full collection

PATTERN	PROBLEM IT SOLVES
Attribute	Many sparse/variable fields → key-value array
Bucket	High-volume time-series → group into time buckets
Outlier	A few docs grow huge → overflow collection
Computed	Expensive aggregates → pre-compute and store
Subset	Large arrays on hot docs → embed only top-N
Extended Reference	Avoid \$lookup for common fields → duplicate stable fields
Approximation	High-write counters → write every N events
Tree	Hierarchical data → ancestors array
Polymorphic	Mixed shapes in one collection → type discriminator

ANTI-PATTERNS	WHAT GOES WRONG
Unbounded arrays	Hits 16MB BSON limit
Over-normalizing	Requires many \$lookup joins – slower reads
Too many indexes	Slows down writes, wastes RAM
Separating related	Forces joins for every read
Storing all in one	Massive document, slow reads, 16MB risk

EMBED WHEN	REFERENCE WHEN
Always accessed together	Accessed independently
Owned by one parent	Shared across many documents
One-to-few relationship	One-to-many or M2M
Small, bounded data	Large or unbounded data
Read performance critical	Write/update performance critical

Guide covers MongoDB 4.4 – 7.x | Schema design is query-driven — always start from your access patterns